

금융 보안 AI



쓰디쓴커피

팀장 : 김진명

팀원 : 박상준

박주영

임태규

목 차

1. 프로젝트 개요 및 문제 접근법

- 1) 생성형 AI 모델 설계 전략 및 접근법
- 2) 기술적 차별성
- 3) 개발 환경 및 도구 구성

2. 상세 개발 방법론

- 1) 생성형 AI 모델 구조 상세
- 2) 데이터 전처리 및 입력 프롬프팅 전략
- 3) 응답 품질 향상을 위한 여러 가지 전략

3. 모델 구현 상세 설명

- Context 구축
- 추론 과정
 - ① CSV 로드 & 유사도 비교
 - ② 질문 증강
 - ③ Context 검색 (1차 검색 / 2차 검색)
 - ④ 프롬프트 빌드 & 생성
 - ⑤ 후처리

4. 결론 및 향후 발전 방향

5. 사용 모델 라이선스

1. 프로젝트 개요 및 문제 접근법

1) 생성형 AI 모델 설계 전략 및 접근법

-모델 선정 이유 및 결정 과정 (최근 한국어 벤치마크에서 높은 점수를 기록한 모델들로 선정)

• A.X-4.0-Light (LLM)

- 한국어 특화 모델로 금융보안 도메인 용어에 이해도 우수.
- RAG와 높은 호환성: 외부 지식 컨텍스트를 반영하여 응답 품질을 크게 향상.
- Long Context Handling: 최대 16,384 토큰 처리 가능
→ 약 300~400쪽 분량의 문서를 다룰 수 있는 규모. 향후 더 커질 지식베이스를 고려하여 모델 선정

• dragonkue/Snowflake Arctic Embed v2.0-ko (임베딩)

- 한국어 특화 문장 임베딩 모델.
- 금융보안 도메인 용어 검색 품질 및 의미 유사도 기반 검색 성능 강화.

• dragonkue /BGE Re-ranker v2 m3-ko (리랭커)

- Cross-Encoder 기반 정밀 리랭킹.
- FAISS 검색의 한계를 보완, 정답 근거의 상위 노출 확보.

• FAISS (벡터 DB)

- 저지연·대규모 벡터 검색 지원.

FSKU 객관식 대응: 단일 정답 강제 프롬프트 + 후처리로 정답률 향상.

FSKU 주관식 대응: 간결·정확한 서술을 유도하는 프롬프트 설계.

공통 대응: cosine유사도 점수가 0.5이상인 Top-k후보 50개를 가져오고 이 중에서 reranking으로 relevance 점수 0.85가 넘는 최대 5개의 context를 프롬프트에 넣어주면서 문제에 대응

※ 최종적으로 추론환경에 맞게 모델을 설계하기 위해 위와 같은 조합을 선택

(RTX 4090 기준 20GB정도의 vram을 사용, 디스크용량 30GB정도 차지)

2) 기술적 차별성

같은 문제로 모델을 설계하다보면 결국 비슷한 양상(embedding, rag, reranking)으로 개발될 것이라고 생각했기에 **context 구축 알고리즘에 차별화**하였음.

같은 문제를 푼다는 것은 비슷한 모델을 개발할 수 있다는 점도 있지만 최종적으로 보면 수 많은 데이터들을 한번에 얻을 수 있다고 생각함.

(대회 주최측 측면에서)

1. 데이콘 토크 게시물과 여러 자료들을 참고하면서 RAG에 필요한 context는 크게 두개(q,a쌍, 본문 chunking)로 구성됨을 알 수 있었음. 결국, 대회 주최측이 가질 데이터셋을 고려하여 **모델 알고리즘은 유지하되 모아진 지식데이터만으로 develop**할 수 있게 코드를 작성.

```
def _clean_str(x):
    s = str(x or "").strip()
    return "" if s.lower() in {"nan", "none", "null"} else s

def build_rag_context(row):
    question = _clean_str(row.get("question", ""))
    answer = _clean_str(row.get("answer", ""))
    context_raw = row.get("context")

    # ● 질문이 비어있으면→ 정답을 본문으로 즉, answer를 context취급
    if not question:
        return f"본문: {answer}" if answer else "본문: "

    # ◆ context 여부에 따라 분기 context가 있으면 context만 쓰고, 없으면 question + answer를 context취급
    context = _clean_str(context_raw)
    if not context:
        return f"질문: {question}\n정답: {answer}"
    else:
        return f"본문: {context}"
```

현재 csv열은 context, question, answer으로 구성되어있음.

추후 모아질 데이터를 위해 3가지 경우에 대비해서 작성.

(*우리팀은 context만 가지고있어 Case 1에 해당)

< 사용 설명 >

Case 1) context만 있을때는 answer열을 context으로 취급하여 csv에 이어 붙임.

Case 2) q,a 일때는 q,a에 맞게 csv에 이어 붙임

Case 3) context,q,a가 모두 있을때는 각 column에 맞게 csv에 이어 붙임.

이렇게하면 전체 알고리즘은 유지한채로 모아진 데이터들을 단순히 csv에 이어붙이는 형식으로 쉽게 모델을 develop 할 수 있게됨.

각 청크길이가 차이나면 문제가 될 수 있지만 그것을 감안하여 임베딩 단계에서

1. question + answer만을 임베딩

2, normalization 진행.

2. 보안 데이터의 경우 허깅페이스의 위키피디아 데이터셋으로 구축되었기 때문에 추가적인 데이터 구축에 단어 하이퍼파라미터만을 추가해서 쉽게 모델을 develop할 수 있음.

3. 추론환경이 4090인 점을 보았을 때 높은 컴퓨팅자원이 확보되지 않음을 알 수 있음. 그렇기에 RAG의 장점을 최대한 살려야한다고 생각했음. 즉, **LLM을 fine-tuning하지 않고 성능을 올렸다는 것**이 우리 알고리즘의 장점.

4. 더 정확하고 빠르게 context를 찾기 위해 **이중 단계 검색구조를 채택.**

1단계에서는 FAISS 기반 벡터검색을 수행함. 이때, 여러 pdf들로 구성되어 어쩔 수 없이 context길이가 제 각각인데 이것을 보완하기 위해 벡터를 normalization함. 또한 코사인 유사도 점수가 0.5 미만인 문단은 사전에 제거. 2단계에서 reranker가 받은 문단을 점수화하여 0.85이상인 문단만을 프롬프트에 넣어줌.

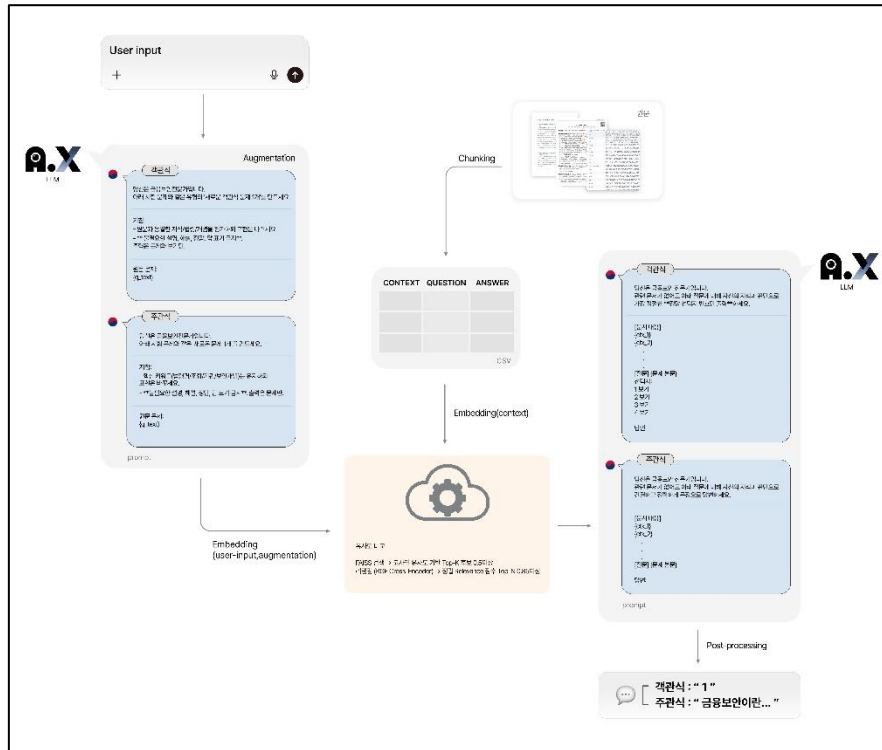
5. 모델 develop을 하려면 하이퍼파라미터 선택을 위해 어쩔 수 없이 코드를 돌려봐야하는데 이것을 빠르게 처리하기 위해 **뽀힌 context가 무엇인지 로그에서 볼 수 있게 코드를 작성**

3) 개발 환경 및 도구 구성

도구/라이브러리	버전	제조사(출처)	용도
Python	3.10	PSF	실행환경
Transformers	4.41.2	Hugging Face	LLM 로딩/추론
Sentence-transformers	3.0.1	UKPLab/Hugging Face	모델 로딩/임베딩
Tokenizers	0.19.1	Hugging Face	토큰나이저
accelerate	<1.0	Hugging Face	추론/ 가속
numpy	1.26.4	Numpy	연산
pandas	최신	Pandas	데이터 처리
tqdm	최신	Tqdm	진행상황
Faiss-cpu	최신	MetaAI	벡터 검색
Huggingface-hub	최신	Hugging Face	모델 다운로드
skt/A.X-4.0-Light	모델 아티팩트	SKT	LLM
drangonkue/bge-reranker-v2-m3-ko	모델 아티팩트	dragonkue	리랭커
drangonkue /snowflake-arctic-embed-l-v2.0-ko	모델 아티팩트	dragonkue	임베딩

2. 상세 개발 방법론

1) 생성형 AI 모델 구조 상세



2) 데이터 전처리 및 입력 프롬프팅 전략

데이터전처리

- 정규화
 - 법률 PDF : pdf 에서 서비스 워터마크/페이지 표기 등 잡음을 제거하고, 각 청크의 머릿말에 [법명] + [위치]를 부여해 문맥을 보존.
 - 위키 백과 : [제목] + [본문]으로 정규화 알고리즘은 법률 문서와 동일하게 적용
- Chunking
 - 장문을 최대 길이로 분할하고, 인접 청크 간 **overlap** 을 두어 경계에서 의미가 끊기지 않게 함. 최소 길이 기준을 뒤 너무 짧은 조각은 다음 윈도우와 병합하여 검색 노이즈를 줄임.

효과: 정제된 청크는 의미 단위로 인덱싱되어 검색 정밀도가 높아지고, 리랭킹·LLM 단계에서 근거를 명확히 제시할 수 있음.

입력 프롬프팅 전략

- **Augmentation**
 - 원문 질문과 동일 유형의 새로운 질문을 LLM 으로 생성해 함께 검색에 사용.
 - 표현 다양성을 확보해 **Recall** 을 높이고, 원본 문제만으로 놓칠 수 있는 후보를 보완.
- **이중 단계 검색**
 - 1 차: 벡터 검색으로 **Top-K** 후보를 회수하고, 유사도 임계값(0.5)으로 빠르게 저품질 context 를 거를 수 있음.
 - 2 차: Cross-Encoder 리랭킹으로 질문,문단 쌍의 **정밀 관련성 점수(0.85)**를 산출해 **Top-N(≤ 5)** 만 최종 컨텍스트로 채택.
- **토큰 처리 방식**
 - 모델의 컨텍스트 한도(16,384 토큰) 맞춰 **Top-N 개수와 각 문단 길이**를 조절.
 - 중복 컨텍스트는 제외해 **핵심 근거 위주**로 입력을 구성.
 - 생성 파라미터는 보수적으로 설정(짧은 답변 유도, 포맷 안정성 확보).
- **객관식 템플릿**
 - 역할 지시("금융보안 전문가") + **컨텍스트(≤ 5)** + 질문을 제공하고,
 - 출력은 정답 선택지 '번호만' 내도록 명확히 지시(설명·근거 출력 금지).
- **주관식 템플릿**
 - 역할 지시("금융보안 전문가") + **컨텍스트(≤ 5)** + 질문을 제공하고,
 - 출력은 간결하고 정확한 단문으로 제한.

3) 응답 품질 향상을 위한 여러가지 전략

임베딩 모델 선택

- **Snowflake Arctic Embed v2.0-ko**(한국어 특화)을 채택. 금융·보안 용어의 의미적 근접도를 안정적으로 반영.
- 임베딩은 **normalization** 을 적용하여 벡터 길이를 1 로 통일함으로써, 이후 내적 연산 점수가 **코사인 유사도**와 동일하게 해석. 또한 길이가 다른 context 를 보완

문서 검색 및 지식 추출 과정 및 하이퍼 파라미터 설정

(1) 쿼리 구성

- 입력 질문은 먼저 **유형 판별(객관/주관)**을 거친 후 검색 Recall 을 높이기 위해 LLM 으로 동일 유형의 '**증강 질문**'을 생성하여, **원문 질문, 증강 질문**을 함께 검색 쿼리로 사용.

(2) 1 차 검색(벡터 검색 & 필터링)

- FAISS 에서 **Top-K(≤ 50)** 후보 문단을 회수.
- 코사인 유사도 **0.5 미만** 문단은 사전에 제거하여 저품질 후보를 처리.
- 원문/증강 쿼리 결과를 **합집합**으로 묶고 **중복 제거**하여 다양한 후보를 확보.

(3) 2 차 검색(리랭킹)

- 후보 문단은 **질문-문단 쌍**으로 Cross-Encoder 에 입력.
- bge-reranker-v2-m3-ko 로 관련성 점수를 산출하고, **0.85 임계값** 이상만 남김.
- 그중 **상위 Top-N(≤ 5)** 를 최종 컨텍스트로 채택.

(4) 컨텍스트 주입과 생성

- 선별된 Top-N 문단을 **프롬프트**에 주입.
- 객관식 템플릿은 정답 '**번호만**' 출력하도록, 주관식 템플릿은 **간결·정확 단문**으로 답하도록 설계.
- 모델은 생성 파라미터($\text{temperature}=0.1, \text{top_p}=0.8$)으로 설정하여 보수적이고 일관적으로 응답.

3. 모델 구현 상세 설명

- Context 구축

(*코드를 일부분만 발췌할 수 없어 데이터전처리.ipynb에 마크다운으로 설명 및 주석으로 설명)

Context 구축 단계에서는 여러 출처의 문서를 통합하여 **질문과 가장 관련성 높은 문맥을** 검색할 수 있도록 설계.

1. 문맥 유지 중심의 Chunking : 단순히 문서를 분할하는 것이 아니라, 어느 법/개념에서 나온 문단인지를 추적할 수 있도록 메타정보를 부여.따라서 [출처명] + [위치정보] 또는 [제목] 등을 청크의 헤더로 붙여, 검색 시 맥락을 보존.
2. 토큰 제한 고려 : 임베딩 모델인 dragonkue/Snowflake Arctic Embed v2.0-ko 의 입력 토큰 한계를 고려하여,min_len, max_len, overlap 을 설정.

이를 통해 **너무 짧은 청크는 병합, 너무 긴 청크는 분할**, 그리고 **경계 부분은 overlap** 으로 보완하여 의미 단절을 최소화.

- 추론

① csv 로드 & 유사도 비교

```
# ✅ GPU 명시 + normalize_embeddings
embed_model = SentenceTransformer(
    "local_models/snowflake-arctic-embed-l-v2.0-ko"
)

rag_data["combined_text"] = (
    "질문: " + rag_data["question"].fillna("") + "\n" +
    "정답: " + rag_data["answer"].fillna("")
)

document_texts = rag_data["combined_text"].astype(str).tolist()
document_embeddings = embed_model.encode(
    document_texts,
    show_progress_bar=True,
    normalize_embeddings=True
).astype("float32")

# ✅ FAISS index (Inner Product)
index = faiss.IndexFlatIP(document_embeddings[0].shape[0])
index.add(np.array(document_embeddings))
print("✅ 질문 임베딩 인덱스 구축 완료")
```

question과 answer을 결합한 텍스트를 임베딩 대상으로 설정. 이유는 앞에서 설명했듯이 context구축과 임베딩 전략을 위함. 또한, 임베딩 벡터는 normalization을 거쳐 단위 벡터로 변환했으며, 이를 통해Inner Product 점수가 코사인 유사도와 동일하게 해석되도록 설계. 인덱싱에는 FAISS IndexFlatIP를 사용하여 코사인 유사도 기반의 Top-K 검색을 안정적으로 수행. 이러한 방식은 특히 스코어 해석과 임계값에 일관성을 제공하여, 1차 후보군을 빠르게 확보하는 데 적합.

② 질문 증강

```
from sentence_transformers import CrossEncoder
# =====
# / LLM 기반 '동일 유형 문제 1개' 생성 유틸
# =====
def _make_augmentation_prompt(q_text: str, is_mc: bool) -> str:
    if is_mc:
        return f"""
당신은 금융보안전문가입니다. 아래 시험 문제와 같은 유형의 '새로운 객관식 문제 1개'를 만드세요.
지침:
- 원문과 동일한 지식/법령/개념을 평가하되 표현은 바꾸세요.
- **불필요한 설명, 해설, 정답, 답 표기 금지**.
```

기존 input만으로 검색을 수행할 경우 표현의 제약으로 인해 관련 문서를 충분히 회수하지 못할 가능성이 존재했음. 이를 해결하기 위해, 원문 문제와 동일한 지식 요소를 평가하지만 표현을 다르게 한 새로운 질문을 LLM을 활용하여 생성한 다음 최종적으로 **원문 질문과 증강 질문을 모두 임베딩하여 검색 쿼리로 활용**. 이를 통해 검색 단계에서 더 다양한 표현을 반영할 수 있었으며, 결과적으로 RAG의 문서 회수율 Recall이 개선됨.

③ context 검색

```
reranker = CrossEncoder("local_models/bge-reranker-v2-m3-ko", device="cuda")
similarity_threshold = 0.5
reranker_threshold = 0.85
top_n = 5
```

```
# ◆ 검색용: 원문 + 동일 유형 문제 1개
sim_query = llm_make_similar_question(test_q, pipe=pipe, tokenizer=tokenizer)

candidate_rows = []
for qv in [test_q, sim_query]: # ← 두 질문 다 검색에 사용
    q_vec = embed_model.encode([qv], normalize_embeddings=True).astype("float32")
    k = int(min(50, max(1, index.ntotal)))
    D, I = index.search(np.ascontiguousarray(q_vec), k=k)
    pairs = [(int(i), float(d)) for i, d in zip(I[0], D[0])
              if i != -1 and 0 <= i < len(rag_data)]
    if similarity_threshold is not None:
        pairs = [(i, d) for (i, d) in pairs if d >= similarity_threshold]
    candidate_rows.extend([rag_data.iloc[i] for i, _ in pairs])
```

1 차 검색 (벡터 검색 및 필터링)

- **Top-K ≤ 50**: 입력 질문(원문 및 증강 질문)을 벡터화하여 FAISS IndexFlatIP 에 질의하고, 최대 50 개의 후보군을 빠르게 회수

- **Similarity_threshold 0.5** : 코사인 유사도 점수가 0.5 미만인 문단은 관련성이 낮다고 판단하여 제거한다. 이 과정을 통해 불필요한 후보가 이후 단계로 전달되지 않도록 하여 계산 효율을 확보.
- **중복 제거**: 동일 문단이 중복 회수되는 경우 하나만 유지하여, 리랭킹 단계가 더 다양한 문단을 공정하게 평가.

```
rerank_inputs = [[full_input, build_rag_context(row)] for row in candidate_rows]
rerank_scores = reranker.predict(rerank_inputs, batch_size=8)

ranked = sorted(zip(candidate_rows, rerank_scores), key=lambda x: x[1], reverse=True)
seen_ids, picked = set(), []
for row, sc in ranked:
    if sc < reranker_threshold:
        continue
    doc_id = row.get("doc_id") if "doc_id" in row else row.name
    if doc_id in seen_ids:
        continue
    picked.append((build_rag_context(row), float(sc), doc_id))
    seen_ids.add(doc_id)
    if len(picked) >= top_n:
        break

if not picked:
    rag_context = ""
else:
    ctx_blocks = [c for c, _, _ in picked]
    rag_context = "\n\n---\n\n".join(ctx_blocks)

# ✅ 로그
print(f"\n🟡 질문 {idx+1}:")
print(test_q)
print("   • 검색용 유사 문제:")
print("       " + sim_query.replace("\n", "\n    "))
if picked:
    for i, (ctx, score, _) in enumerate(picked, 1):
        print(f"   • [선택 {i}] reranker score: {score:.4f}")
        print(f"   {ctx[:300]}...\n")

# 💎 최종 프롬프트 (답변에는 원문만 사용)
prompt = make_prompt_rag(test_q, context=rag_context)
output = pipe(prompt, max_new_tokens=400, temperature=0.1, top_p=0.8,
              pad_token_id=tokenizer.eos_token_id, eos_token_id=tokenizer.eos_token_id)

pred_answer = extract_answer_only(output[0]["generated_text"], original_question=test_q)
preds.append(pred_answer)
```

2 차 검색 (reranking)

- **0.85 Threshold 적용**: bge-reranker-v2-m3-ko 모델을 이용해 질문-문단 쌍의 관련성을 점수화. 점수가 0.85 이상인 문단만 최종적으로 선택하여, 이후 LLM 응답 생성을 위한 컨텍스트로 사용.
- **상위 Top-N (≤ 5)**: 필터링을 통과한 후보들 중 상위 5 개를 최종 컨텍스트로 채택하여, LLM 에 제공되는 정보가 지나치게 방대하지 않으면서도 충분한 근거를 담도록 설계.

④ 프롬프트 빌드 & 생성

```
def is_multiple_choice(question_text):
    lines = question_text.strip().split("\n")
    option_count = sum(bool(re.match(r"^\s*[1-9][0-9]?\s", line)) for line in lines)
    return option_count >= 2

def extract_question_and_choices(full_text):
    lines = full_text.strip().split("\n")
    q_lines, options = [], []
    for line in lines:
        if re.match(r"^\s*[1-9][0-9]?\s", line):
            options.append(line.strip())
        else:
            q_lines.append(line.strip())
    question = " ".join(q_lines)
    return question, options
```

```
if is_multiple_choice(text):
    question, options = extract_question_and_choices(text)
    prompt = (
        "당신은 금융보안 전문가입니다.\n"
        "관련 문서가 없어도 아래 질문에 대해 자신의 지식과 판단으로 가장 적절한 **정답 선택지 번호만 출력**하세요.\n\n"
        f"[문서 내용]\n{context}\n\n"
        f"[질문] {question}\n"
        "선택지:\n"
        f"{chr(10).join(options)}\n\n"
        "답변:"
    )
else:
    prompt = (
        "당신은 금융보안 전문가입니다.\n"
        "관련 문서가 없어도 아래 질문에 대해 자신의 지식과 판단으로 간결하고 정확하게 문장으로 답변하세요.\n\n"
        f"[문서 내용]\n{context}\n\n"
        f"[질문] {text}\n\n"
        "답변:"
    )
return prompt
```

```
# ◆ 최종 프롬프트 (답변에는 원문만 사용)
prompt = make_prompt_rag(test_q, context=rag_context)
output = pipe(prompt, max_new_tokens=400, temperature=0.1, top_p=0.8,
              pad_token_id=tokenizer.eos_token_id, eos_token_id=tokenizer.eos_token_id)

pred_answer = extract_answer_only(output[0]["generated_text"], original_question=test_q)
preds.append(pred_answer)
```

먼저 입력 질문을 `is_multiple_chioce()`로 객관식 문제인지 주관식 문제인지를 구분하고, 그에 맞는 지시문과 출력 형식을 프롬프트에 반영. 객관식 문제의 경우, 모델이 불필요한 설명을 출력하지 않고 **정답 선택지 번호만 산출**하도록 지시. 이를 위해 "당신은 금융보안 전문가입니다"라는 역할 기반 문구와 함께, 주어진 문서 내용과 질문, 그리고 선택지를 구조적으로 나열하고 마지막에 "답변:" 형식을 부여. 주관식 문제의 경우, 모델이 **간결하고 정확한 문장**으로 답변하도록 지시.

최종적으로 완성된 프롬프트는 리랭킹 단계에서 선별된 **상위 Top-N(≤ 5)의 context 문단**과 함께 LLM에 입력되고 모델은 최대 400 토큰 내에서 답변을 생성하며, temperature 0.1, top-p 0.8과 같은 보수적인 생성 파라미터를 적용하여 안정적이고 일관된 답변을 출력.

⑤ 후처리

```
def postprocess_answer(ans: str, original_q: str) -> str:
    """객관식은 숫자만 안전 추출, 주관식은 */공백/줄바꿈 정리"""
    s = str(ans or "")

    if is_multiple_choice(original_q):
        m = re.search(r'([1-9][0-9]?)', s) # 1~99까지 허용
        return m.group(1) if m else "0"

    # --- 주관식 정리 ---
    s = unicodedata.normalize("NFKC", s) # 유니코드 정규화
    s = s.replace("***", "").replace("__", "") # 마크다운 굵게 제거
    s = re.sub(r'`{1,3}', '', s) # 인라인 코드 백틱 제거

    # 줄바꿈/공백 정리
    s = s.replace("\r\n", "\n").replace("\r", "\n")
    s = "\n".join(line.strip() for line in s.splitlines()) # 라인별 좌우공백 제거
    s = re.sub(r'[ \t]+', ' ', s) # 여러 칸 공백 → 1칸
    s = re.sub(r'\n{3,}', '\n\n', s) # 과도한 빈 줄 축약
    s = s.strip(' \"\'\"') # 바깥쪽 따옴표/공백 제거
    return s
```

LLM의 최종답변을 사용자가 보기 편하게끔 후처리를 하여 최종 output을 출력.

4. 결론 및 향후 발전 방향

모델 리더보드에서 상위권에 위치해 있지만 실무에 즉시 적용하기에는 아직 완벽하지 않은 것이 사실이다. 다만, 앞서 언급한 것처럼 **여러 지식 데이터를 결합하고 알고리즘을 최적으로 수정해서 지식베이스를 확장·발전**시키는 방향이 더욱 바람직하다. 이것을 위해 우리 코드는 **빠른 하이퍼파라미터 조정과 fine-tuning 없는 구조**에 있으며, 이는 최신성이 중요한 **법률·보안 데이터**에 특히 적합하다. 또한 추론 환경을 최적화하고 자원을 극대화한다면, 본 모델 역시 금융보안 실무에서 충분히 활용 가능할 것이라 자신한다.

5. 사용 모델 라이선스

- **LLM**: SKT, A.X-4.0-Light (Apache-2.0)
- **리랭커**: dragonkue, bge-reranker-v2-m3-ko (Apache-2.0)
- **임베딩 모델**: dragonkue, snowflake-arctic-embed-l-v2.0-ko (Apache-2.0)
- **Transformers**: Hugging Face, Transformers (Apache-2.0)
- **Sentence-Transformers**: UKPLab/Hugging Face (Apache-2.0)
- **FAISS**: Meta AI, FAISS (MIT License)
- **Datasets**: wikimedia/wikipedia (cc-by-sa-3.0)